

Supporting Information: Quasiclassical Trajectory and Quantum-Chemistry Interface Features in Smite

1 Overview

Smite is a Python toolkit for direct quasiclassical trajectory (QCT) and molecular dynamics calculations on either analytic/model potential energy surfaces (PESs), machine-learning-style PES wrappers, or on-the-fly electronic-structure gradients. The public simulation objects are **Fragment**, **Collision**, and **Molecule**. A **Fragment** stores atom labels, masses, Cartesian coordinates, momenta, normal-mode data, and sampling settings. A **Collision** combines two fragments and adds bimolecular sampling variables such as initial center-of-mass separation, impact parameter, relative collision energy, and stopping conditions. A **Molecule** provides the common trajectory propagator used for unimolecular and bimolecular dynamics.

All internal propagation uses atomic units. User-facing molecular geometries are supplied in Angstrom and converted to bohr. Printed trajectories use Angstrom for coordinates and atomic units for momenta. Energies are stored in hartree internally and are reported with common conversions to kJ mol^{-1} , kcal mol^{-1} , and cm^{-1} where appropriate.

2 Supported QCT Workflows

2.1 Molecular systems

The QCT layer supports:

- atom, diatom, and polyatom fragments;
- nonrigid normal-mode sampling for polyatomic molecules;
- harmonic and Morse oscillator sampling for diatomics;
- rigid fragments and surface fragments;
- unimolecular trajectories from semiclassical initial conditions;
- bimolecular gas-phase collision trajectories;
- molecule-surface collision geometries with incidence-angle sampling;
- restart-style trajectory backup files;
- optional post-trajectory spectral, scattering, and vector-correlation analysis;
- independent parallel trajectory execution with per-trajectory output and scratch directories.

2.2 Trajectory propagation

The central propagation routine evaluates the potential energy and Cartesian forces at the current geometry, advances coordinates and momenta, optionally applies a thermostat, writes trajectory frames, and checks termination criteria. Available propagators are:

- velocity Verlet;
- leapfrog;
- Stormer–Verlet;
- fourth-order Runge–Kutta;
- fourth-, sixth-, and eighth-order symplectic factorizations;
- symplectic partitioned Runge–Kutta;
- predictor–corrector integration.

Constant-energy NVE trajectories are obtained when no thermostat is selected. NVT-style segments are available through Berendsen, Andersen, Nose–Hoover, and generalized Langevin equation (GLE) thermostats. The trajectory temperature is computed from

$$T = \frac{2K}{N_{\text{dof}}R}, \quad (1)$$

where $K = \sum_i p_i^2/(2m_i)$, N_{dof} is the number of active Cartesian degrees of freedom after subtracting any fixed degrees of freedom, and R is the gas constant in hartree K^{−1}.

3 Semiclassical Quantization of Initial Conditions

3.1 Polyatomic vibrational sampling

For nonrigid polyatomic fragments, Smite obtains a Hessian, projects out translations and rotations through the normal-mode machinery, and samples vibrational normal modes. The normal-mode matrix maps mass-weighted mode displacements into Cartesian coordinates,

$$\mathbf{q} = \mathbf{q}_{\text{eq}} + \mathbf{L}\mathbf{Q}. \quad (2)$$

The inverse diagnostic projection is

$$Q_k = \mathbf{L}_k^T \mathbf{M}(\mathbf{q} - \mathbf{q}_{\text{eq}}), \quad \dot{Q}_k = \mathbf{L}_k^T \mathbf{p}, \quad (3)$$

and the recovered harmonic mode energy is

$$E_k = \frac{1}{2}\dot{Q}_k^2 + \frac{1}{2}\omega_k^2 Q_k^2. \quad (4)$$

Each normal mode can be sampled independently using one of four modes:

- Q: fixed vibrational quantum number n_k , with

$$E_k = \omega_k \left(n_k + \frac{1}{2} \right); \quad (5)$$

- **E**: fixed mode energy, allowing a noninteger effective quantum number

$$n_k^{\text{eff}} = \frac{E_k}{\omega_k} - \frac{1}{2}; \quad (6)$$

- **T**: thermal vibrational sampling, where a quantum number is sampled from a Boltzmann-like distribution by

$$n_k = \left\lfloor \frac{-RT \ln(1 - \xi)}{\omega_k} \right\rfloor; \quad (7)$$

- **W**: ground-state Wigner sampling of the harmonic oscillator.

Here ξ is a uniform random number in $(0, 1)$, and RT is expressed in hartree. For low-frequency thermal sampling a frequency cutoff of 100 cm^{-1} is used for the sampling distribution while retaining the actual frequency in the final energy.

For the fixed-energy QCT modes, a uniformly random phase $\phi_k \in [0, 2\pi)$ is selected and the normal-coordinate amplitude is

$$A_k = \frac{\sqrt{2E_k}}{\omega_k}. \quad (8)$$

The sampled mode displacement and velocity are then

$$Q_k = A_k \cos \phi_k, \quad \dot{Q}_k = -A_k \omega_k \sin \phi_k. \quad (9)$$

The normal-mode displacements and velocities are transformed back to Cartesian coordinates and momenta. Center-of-mass translation is removed after sampling.

For ground-state Wigner sampling, the code samples the harmonic oscillator Wigner distribution in the convention

$$W_0(Q, P) = \pi^{-1} \exp \left(-\omega Q^2 - \frac{P^2}{\omega} \right), \quad (10)$$

so that $\text{var}(Q) = 1/(2\omega)$ and $\text{var}(P) = \omega/2$. The sampled energy is converted to an effective quantum number for diagnostics.

Mode sampling is configured globally with `init_vib_type = ZPE, Temp, or Wigner`, and individual modes can be overwritten using lists such as `fix_quantum`, `fix_energy`, `fix_temp`, and `fix_wigner`.

3.2 Diatomic vibrational sampling

Diatomic fragments can be sampled as harmonic oscillators or as Morse oscillators. For the harmonic model, the same **Q**, **E**, **T**, and **W** options are available. Given reduced mass μ , equilibrium bond length r_e , and frequency ω , a fixed quantum number gives

$$E_v = \omega \left(n + \frac{1}{2} \right). \quad (11)$$

A random phase is used to sample the bond displacement and conjugate momentum,

$$\Delta r = \sqrt{\frac{2(n + 1/2)}{\mu\omega}} \cos \phi, \quad p_r = -\sqrt{2(n + 1/2)\mu\omega} \sin \phi, \quad (12)$$

followed by construction of two Cartesian atoms along the molecular axis and removal of center-of-mass motion.

For Morse diatomics, Smite implements a rotating Morse oscillator sampling based on the Porter–Raff–Miller formulation. The user supplies r_e , D_e , and either β or α . The Morse zero-order frequency is

$$\omega_0 = \beta \sqrt{\frac{2D_e}{\mu}}, \quad (13)$$

and the rovibrational energy includes harmonic, anharmonic, centrifugal, and rovibrational coupling terms. The Morse coordinate $\xi = \exp[-\beta(r-r_e)]$ and p_r are sampled from the corresponding phase-space construction. In this model the rotational quantum number enters the vibrational coordinate sampling, so rovibrational sampling is coupled rather than applied as two independent steps.

3.3 Rotational sampling

Rotational excitation is requested through `init_rot_type`. The two implemented options are:

- **Jfix**: a fixed rotational quantum number J ;
- **Temp**: thermal angular momentum or thermal rotational quantum-number sampling.

For fixed- J sampling the angular momentum magnitude is

$$|\mathbf{J}| = \sqrt{J(J+1)} \quad (14)$$

in atomic units. The direction of $\mathbf{J}$ is randomized. For diatomics, the angular momentum lies perpendicular to the molecular axis. For polyatomic fragments, the angular momentum vector is sampled in three dimensions, corrected for any vibrational angular momentum already present,

$$\mathbf{J}_{\text{sample}} = \mathbf{J}_{\text{target}} - \mathbf{J}_{\text{vib}}, \quad (15)$$

and converted to angular velocity by solving $\mathbf{I}\boldsymbol{\omega} = \mathbf{J}_{\text{sample}}$. The rotational momenta are then added atom by atom as

$$\mathbf{p}_i \leftarrow \mathbf{p}_i + m_i(\boldsymbol{\omega} \times \mathbf{r}_i). \quad (16)$$

Thermal polyatomic rotation is sampled with symmetric-top formulas, using prolate, oblate, or near-spherical branches depending on the principal moments of inertia. Thermal diatomic rotation samples J from the spherical-top distribution associated with the moment of inertia.

3.4 Sampling from previous MD

For polyatomic fragments initialized with `fromMD=True`, Smite can read a previous trajectory file containing coordinates and momenta and randomly choose one saved frame as the starting condition. A simpler `MDprimitive` option assigns random momenta at a requested temperature without normal-mode quantization.

4 Bimolecular Relative Initial Coordinates

4.1 Collision-energy sampling

For bimolecular collisions the user specifies the initial center-of-mass separation R_{ini} , maximum impact parameter b_{max} , and either a fixed collision energy or a collision temperature. Fixed collision energies are supplied in kJ mol^{-1} and converted to hartree. Thermal collision energies use the Maxwell–Boltzmann relative translational energy distribution sampled by

$$E_{\text{coll}} = -RT \ln(\xi_1 \xi_2), \quad (17)$$

where ξ_1 and ξ_2 are independent uniform random numbers.

4.2 Impact-parameter options

The impact parameter can be selected in three ways:

- `bsampling=0`: fixed impact parameter, $b = b_{\text{max}}$;
- `bsampling=1`: linear sampling, $b = b_{\text{max}}\xi$;
- `bsampling=2`: area-uniform sampling, $b = b_{\text{max}}\sqrt{\xi}$.

The area-uniform option corresponds to uniform sampling over the collision disk.

4.3 Gas-phase molecule–molecule geometry

Before relative coordinates are assigned, each fragment is shifted to its own center-of-mass frame. Fragment B is placed relative to fragment A with initial center-of-mass distance R_{ini} and impact parameter b :

$$x_{\text{sep}} = \sqrt{R_{\text{ini}}^2 - b^2}. \quad (18)$$

The code translates all atoms of fragment B by x_{sep} along the laboratory x axis and by b along the laboratory z axis. With fragment masses M_A and M_B , reduced mass

$$\mu = \frac{M_A M_B}{M_A + M_B}, \quad (19)$$

and collision energy E_{coll} , the relative speed is

$$v_{\text{rel}} = \sqrt{\frac{2E_{\text{coll}}}{\mu}}. \quad (20)$$

The center-of-mass velocities assigned to the two reactants are

$$v_A = v_{\text{rel}} \frac{M_B}{M_A + M_B}, \quad v_B = v_A - v_{\text{rel}}, \quad (21)$$

and are converted to atomic momenta by multiplying by each atom's mass. The resulting total center-of-mass momentum is zero.

4.4 Surface collision geometry

For molecule-surface collisions one fragment can be marked as a surface. The surface is oriented into the laboratory yz plane so that its normal points along the x axis. The projectile incidence direction is controlled by an incidence/skew angle. If `surf_skew_fix` is not supplied, the skew angle is sampled over a spherical cap using

$$\cos \theta = 1 - \xi (1 - \cos \theta_{\text{max}}), \quad (22)$$

which gives uniform solid-angle sampling up to `surf_skew_max`. The azimuthal direction in the surface plane is also randomized. The projectile is translated to the initial separation and given a velocity directed toward the surface.

4.5 Trajectory stopping criteria

Trajectories can stop by either:

- a general separation threshold `Rstop`;
- a channel-specific list of pair-distance conditions `pairs_to_stop`.

The channel-specific form supports conditions such as “distance between atoms i and j greater than a threshold” or “less than a threshold”, grouped by product-channel label. This is useful for distinguishing nonreactive scattering, abstraction, association, or dissociation products.

5 Quantum-Chemistry and PES Interfaces

5.1 Common input dictionary

All electronic-structure and PES backends use a dictionary-like `qcinput`. The common keys are summarized in Table 1.

Table 1: Common `qcinput` keys used by the force and energy interfaces.

Key	Meaning
<code>qchem</code>	Backend selector. Supported values are listed below the table.
<code>path</code>	Executable path for external binary backends, or PES path/name information.
<code>nproc</code>	Number of processor threads requested for the backend.
<code>functional</code>	Electronic-structure method, density functional, semiempirical method, or xTB method option.
<code>basis</code>	Basis set for backends that require one.
<code>charge</code>	Total molecular charge.
<code>multiplicity</code>	Spin multiplicity, $2S + 1$.
<code>additional</code>	Backend-specific extra keywords or command-line options.
<code>wfu</code>	Whether to save wavefunction/orbital information along the trajectory.
<code>scratch_dir</code>	Runtime scratch directory, usually assigned per trajectory.

The supported `qchem` values are `XTB`, `Orca`, `PySCF`, `Psi4`, `Sparrow_bin`, `Sparrow_Py`, and `PES`.

The input validator normalizes backend aliases such as `orca`, `xtb`, `pyscf`, `psi4`, and `pes`, fills defaults, checks integer charge/multiplicity/thread counts, verifies required executable paths for binary backends, and rejects unsupported backends.

5.2 Energy and force dispatch

The trajectory propagators call a common force routine. For each geometry, Smite dispatches to the selected backend, obtains an energy and/or gradient, and returns Cartesian forces with the convention

$$\mathbf{F} = -\nabla_{\mathbf{q}}V(\mathbf{q}). \quad (23)$$

Supported backends are:

- `XTB`: external xTB executable, with gradient parsing from xTB output;

- **Orca**: ORCA input generation with `engrad`, gradient parsing, and optional orbital restart through previous `.gbw` files;
- **PySCF**: in-process PySCF RKS/UKS DFT energy, gradient, and Hessian calls;
- **Psi4**: in-process Psi4 RKS/UKS energy and gradient calls, with a more robust fallback SCF option for difficult points;
- **Sparrow_bin**: external Sparrow executable;
- **Sparrow_Py**: Python Sparrow interface;
- **PES**: user-provided analytic, fitted, or machine-learning PES interface.

After a force call, the corresponding energy and force can be cached in `qcinput`. If the next energy call is requested at exactly the same coordinates and atom ordering, the cached value is reused. This avoids duplicate backend evaluations in the common velocity-Verlet pattern where a force and then an energy are needed for the same geometry.

5.3 Scratch management and parallel runs

Backend scratch files are isolated through `scratch_dir`. A single trajectory assigns scratch under `scratch/<trajectory-name>/`. Parallel trajectories create per-trajectory output directories such as `run_name/traj_000003/` and scratch directories such as `scratch/run_name/traj_000003/`. Each parallel worker receives its own copy of the molecular system, its own trajectory and backup files, its own initial-condition summary, and its own backend scratch path. External quantum-chemistry backends receive the requested `nproc` per job. PES jobs can be forced to one core by default because many fitted PES calls are not threaded.

Backend helpers check executable paths, capture standard output and standard error, report parse errors with useful context, and can optionally retry a backend after cleaning its scratch directory.

5.4 Wavefunction output

If `wfu=True`, selected backends can write wavefunction information along the trajectory. ORCA wavefunctions are converted from `.gbw` to Molden format through the ORCA `_2mk1` utility. PySCF and Psi4 write Molden-style output directly through their Python interfaces. PES backends reject `wfu=True`, because a classical PES does not produce an electronic wavefunction.

5.5 Hessians for normal-mode sampling and optimization

Hessians are obtained through the same backend abstraction and are cached to text files such as `hessian.<name>.hess`. Existing Hessian files are reused unless forced recalculation is requested. ORCA and PySCF can provide direct Hessian calculations. Psi4, XTB, Sparrow, and generic PES interfaces can provide numerical Hessians by finite differences of analytical gradients or forces when no analytic Hessian method is available. PES interfaces can also expose a direct `hessian(q, atoms)` method.

5.6 Analytic, fitted, and machine-learning PES interface

The generic PES backend is designed for analytic surfaces, fitted surfaces, and machine-learning potentials that can be called from Python. A PES directory contains a `pes_interface.py` file defining a `PESCalculator` class. The calculator must provide

- `energy(q, atoms)`; and
- either `force(q, atoms)` or `gradient(q, atoms)`.

Coordinates are supplied in bohr and atom labels are supplied as the trajectory atom list. If the calculator defines `gradient`, Smite converts it to force by a minus sign. If it defines `hessian`, that Hessian is used directly; otherwise a finite-difference Hessian is available. A separate validation helper checks PES energy, force, and a finite-difference gradient component to verify the sign convention and numerical consistency.

6 Standalone Optimization and Reaction-Path Tools

Although this document emphasizes QCT dynamics and force interfaces, the same backend layer is used by the standalone optimization module. The optimizer package exposes:

- minimum-energy geometry optimization;
- transition-state optimization;
- redundant internal-coordinate optimization;
- RDA-based quasi-TS guess generation from reactant and product endpoint structures;
- spin-crossing optimization using two spin-state gradients;
- intrinsic reaction coordinate following;
- one- and two-dimensional coordinate scans and normal-mode scans.

The optimizer can use the Smite force/Hessian backend or selected external optimizer behavior, depending on the provided `OptimizerConfig`. It therefore supports the same ab initio and PES backends as the trajectory code.

7 Output Files and Diagnostics

Trajectory files are written in an extended XYZ-like format with one frame per saved step. Each frame contains atom labels, coordinates, momenta, potential energy, kinetic energy, energy drift, simulation time, and instantaneous temperature. Backup files store the latest geometry and momenta for restart use. Optional diagnostics include:

- sampled normal-mode energy recovery files;
- vibrational spectra from saved velocity histories;
- time-dependent scattering form factors;
- post-collision vector-correlation analysis;
- per-trajectory parallel status files in JSON format;
- suppressed worker output logs for parallel runs.

8 Representative User-Level Options

At the input-script level, the most important QCT controls are:

- fragment construction: atom, diatom, polyatom, rigid, nonrigid, surface, or from previous MD;

- vibrational sampling: ZPE, Temp, Wigner, per-mode `fix_quantum`, `fix_energy`, `fix_temp`, and `fix_wigner`;
- rotational sampling: `Jfix` or `Temp`;
- diatom model: harmonic or Morse;
- collision setup: `Rini`, `bmax`, `bsampling`, fixed `Ecoll`, or thermal `Ecoll_thermal`;
- surface setup: surface side, maximum or fixed skew/incidence angle, and random azimuthal orientation;
- propagation: integrator name, integrator order, timestep, maximum step count, and print stride;
- thermalization: Berendsen, Andersen, Nose–Hoover, or GLE thermostat with target temperature and coupling parameter;
- stopping conditions: `Rstop` or channel-specific `pairs_to_stop`;
- backend: `XTB`, `Orca`, `PySCF`, `Psi4`, `Sparrow_bin`, `Sparrow_Py`, or `PES`;
- parallelism: total trajectories, number of simultaneous jobs, processors per job, output directory, scratch directory, and error policy.